

Angular Moderno

10 - Effects

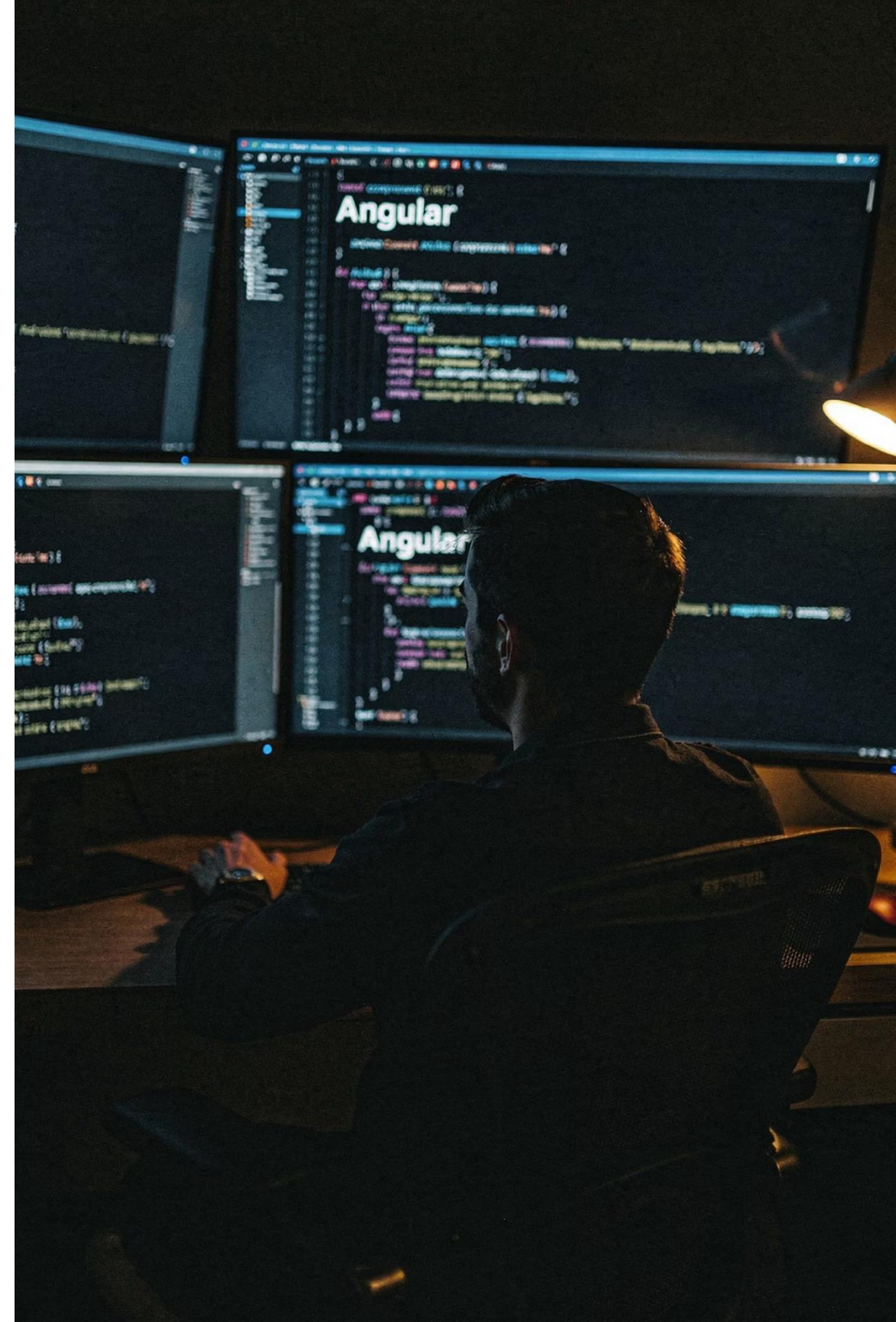
2026

SENAI - CRTI

10 — Effects

Reações automáticas no Angular moderno

Nesta aula você vai aprender como o Angular pode **reagir automaticamente** a mudanças de estado, integrando a lógica reativa do seu projeto com o mundo externo.



O que vamos aprender hoje

01

O que é **effect()**

Entender o conceito de efeito colateral reativo no Angular moderno.

03

Reagir a mudanças automaticamente

Criar effects que disparam ações sempre que o estado do seu projeto é alterado.

02

Signal, Computed e Effect

Compreender a diferença fundamental entre os três pilares da reatividade.

04

Aplicação prática no projeto

Implementar effects reais no e-commerce: logs, título da aba e seleção de produto.

Onde estamos no projeto

Antes de avançar, veja o que já foi construído até aqui no e-commerce:

Lista de Produtos

Componente `lista-produtos` (pai) renderizando `produto` (filho) com `@for`.

Estado com `signal()`

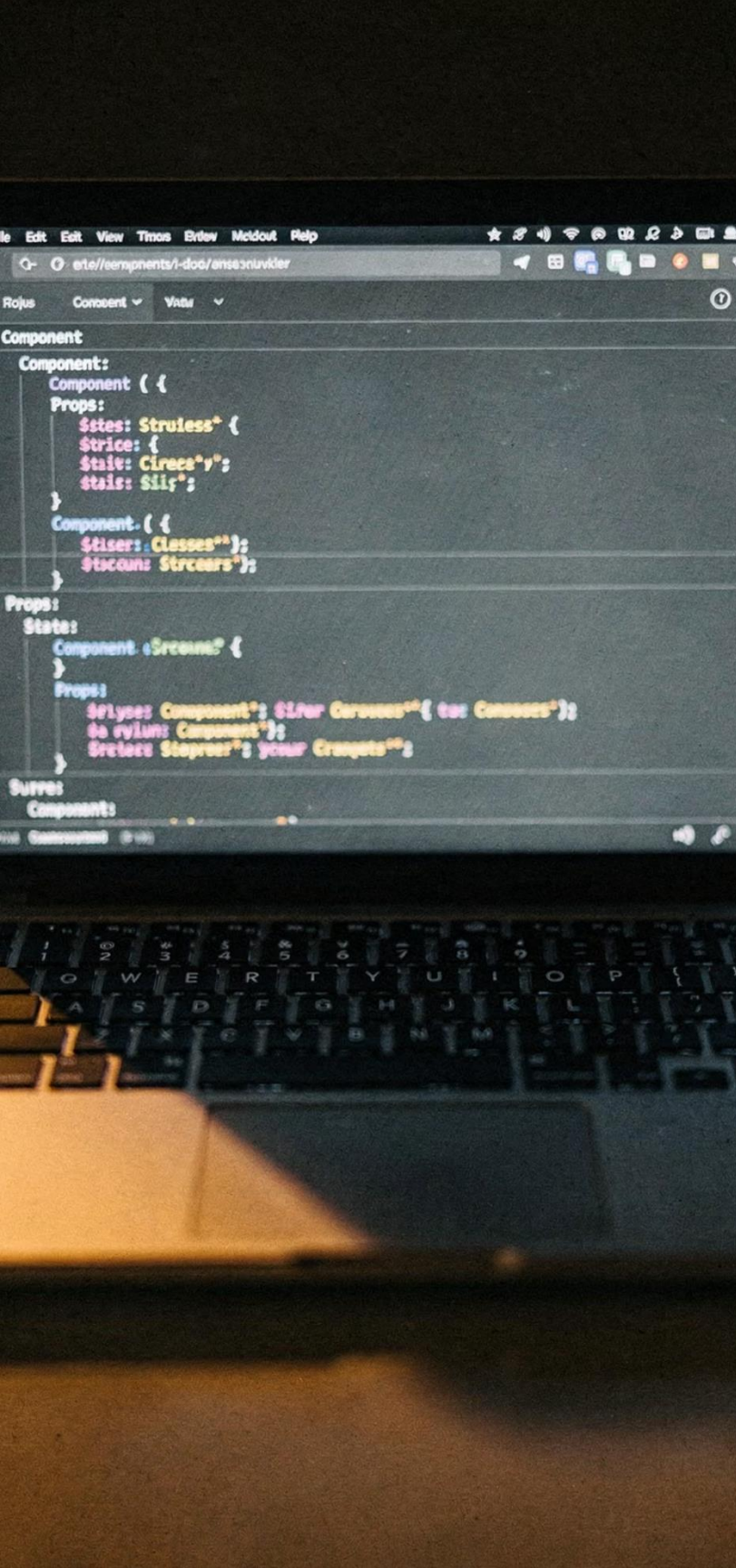
Estado reativo gerenciado com `signal()`, atualizado via métodos do componente.

Cálculos com `computed()`

Valores derivados calculados automaticamente, como total de produtos e valor total.

Interface Reativa

O template atualiza sozinho sempre que o estado muda.



O problema atual

O que já funciona

Você já tem estado reativo com `signal()` e cálculos automáticos com `computed()`. A interface responde perfeitamente às mudanças no template.

- Estado atualiza o template
- Computed recalcula valores
- Interface reage visualmente

O que ainda falta

Porém, você ainda não consegue **reagir a mudanças fora do template**. Há situações comuns que exigem essa capacidade:

- Registrar logs no console
- Atualizar o título da aba do navegador
- Integrar com APIs externas
- Disparar ações em serviços

❗ É exatamente para isso que existe o **effect()**. Ele conecta a reatividade interna do Angular com o mundo externo.

O que é `effect()`?

O `effect()` é uma função do Angular que **executa automaticamente sempre que um signal observado muda de valor**. Ele não retorna nenhum dado. Sua função é exclusivamente causar um efeito colateral.



Executa automaticamente

Você não precisa chamar o `effect` manualmente. Ele roda sempre que os signals que ele observa mudam.



Rastreia dependências

O Angular detecta quais signals são lidos dentro do `effect` e os observa automaticamente, sem configuração extra.

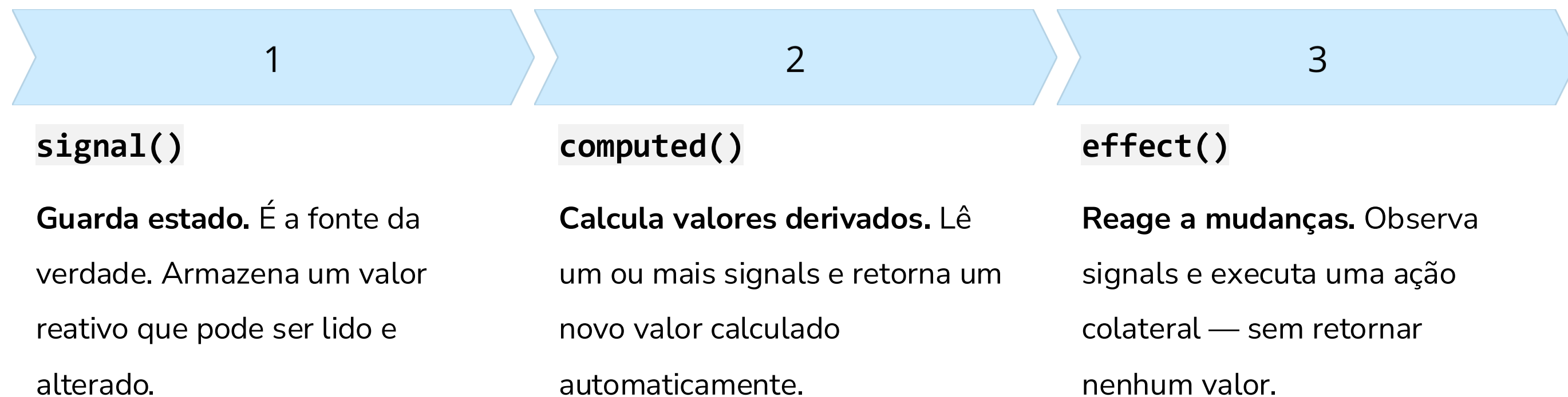


Para efeitos colaterais

Use-o para logs, atualização de título e chamadas externas. Nunca para derivar ou calcular novos valores.

A diferença fundamental

Os três pilares da reatividade no Angular moderno têm papéis distintos e complementares. Entender essa diferença é essencial para escrever código limpo:



📌 Lembre-se: `computed` produz um valor. `effect` produz uma ação. Nunca os confunda.

Passo 1 — Parar o servidor

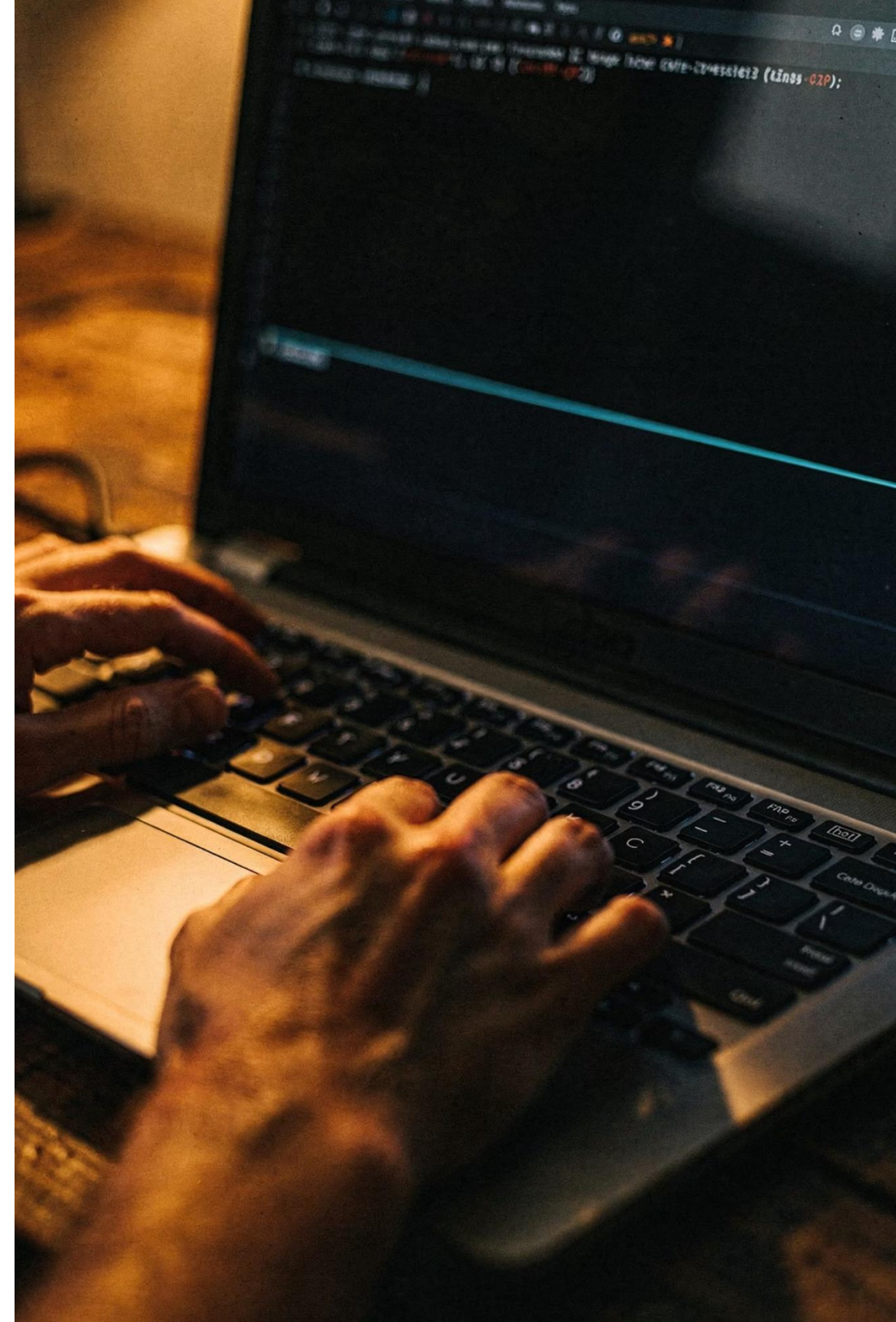
Antes de modificar qualquer arquivo do projeto, você precisa parar o servidor de desenvolvimento. Isso evita erros de compilação durante a edição.

No terminal, pressione:

CTRL + C

Aguarde a mensagem de encerramento aparecer. O terminal ficará disponível para novos comandos.

⚠ **Apesar de não ser uma exigência**, é interessante parar o servidor antes de editar arquivos de TypeScript. Alterações com o servidor rodando podem gerar erros de compilação difíceis de depurar.



Passo 2 — Importar **effect**

Abra o arquivo `lista-produtos.ts` e adicione `effect` à importação do `@angular/core`. Você só precisa mudar uma linha:

Antes

```
import { Component, signal, computed }  
from '@angular/core';
```

Depois

```
import { Component, signal, computed, effect }  
from '@angular/core';
```

 Adicionar `effect` ao import é suficiente — ele já faz parte do núcleo do Angular.

Nenhuma outra configuração é necessária.

Passo 3 — Criar o primeiro **effect**

Dentro da classe do componente `lista-produtos`, crie um método `constructor`, adicione o seguinte effect para observar a lista de produtos:

```
constructor() {  
  effect(() => {  
    console.log('Lista de produtos alterada:', this.produtos());  
  });  
}
```

O que acontece aqui?

O Angular lê `this.produtos()` durante a execução do effect e registra **automaticamente** esse signal como dependência. A cada mudança na lista, o callback é executado novamente, **sem que você precise configurar nada**.

❏ O **effect** deve sempre ser criado dentro do `constructor` ou no contexto de injeção do componente.

Passo 4 — Criar o segundo `effect`

Agora adicione um segundo effect logo abaixo do primeiro, ainda dentro do `constructor`. Dessa vez, vamos observar o `computed` de valor total:

```
effect(() => {  
  console.log('Valor total atualizado:', this.valorTotal());  
});
```

Por que isso funciona?

O `computed()` também é um signal de leitura. O `effect()` consegue observá-lo da mesma forma que observa um `signal()` comum. Sempre que o valor calculado mudar, o effect dispara. O `computed()` é reativo!

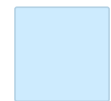
Resultado esperado

No console do navegador, sempre que um produto for adicionado ou removido, você verá **ambos os logs** sendo disparados automaticamente.

Detecção automática de dependências

Zero configuração manual

Um dos pontos mais poderosos do `effect()` é que você **não precisa declarar quais signals ele observa**. O Angular rastreia isso automaticamente em tempo de execução.



Como funciona na prática

Durante a primeira execução do `effect`, o Angular registra todos os signals que foram **lidos** (chamados com `()`). Esses signals tornam-se as dependências do `effect`.



O que dispara uma nova execução

Sempre que qualquer um dos signals rastreados mudar de valor, o Angular agenda uma nova execução do `effect` automaticamente, na próxima verificação de mudanças.

Passo 5 - Caso prático — Título da aba do navegador

Agora vamos criar um effect que integra o Angular diretamente com o navegador: atualizar o título da aba em tempo real conforme produtos são adicionados. Inclua isso ao final do método `constructor()`:

```
effect(() => {  
  if (typeof document !== 'undefined') {  
    document.title = `(${this.totalProdutos()}) Minha Loja`;  
  }  
});
```

Não são aspas simples.

São CRASES!

A crase (`) permite inserir valores dentro do texto usando \${ }.

Por que usar `typeof document !== 'undefined'`?

Durante a compilação ou em ambientes de renderização no servidor (SSR), o objeto `document` não existe. Essa verificação protege o código de erros em contextos fora do navegador.

O que você vai ver

A aba do navegador exibirá, por exemplo, **(3) Minha Loja** sempre que houver 3 produtos na lista. Isso será atualizado automaticamente!

Passo 6 — Reiniciar o servidor

Com todas as alterações salvas, é hora de reiniciar o servidor de desenvolvimento para compilar o código atualizado.

No terminal, execute:

```
ng serve
```

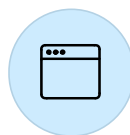
Aguarde a mensagem `Application bundle generation complete` antes de abrir o navegador. Isso confirma que a compilação foi bem-sucedida.

- ✔ Se não houver erros no terminal, suas alterações foram compiladas corretamente e o projeto está pronto para ser testado.



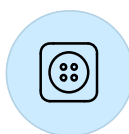
Passo 7 — Testar a aplicação

Com o servidor rodando, abra o navegador em `http://localhost:4200` e siga estes passos para verificar se os effects estão funcionando:



Abra o DevTools

Pressione `F12` e vá até a aba **Console** para visualizar os logs em tempo real.



Adicione produtos

Clique no botão para adicionar produtos à lista. Observe o console a cada adição.



Verifique o título da aba

Olhe para a aba do navegador. Ela deve exibir **(1) Minha Loja**, **(2) Minha Loja** etc., atualizando a cada produto adicionado.

O que aconteceu por baixo dos panos



Estado mudou

Computed recalculou

Effect reagiu

Interface atualizada

Você acabou de ver o ciclo completo da reatividade do Angular em ação: uma mudança no estado se propagou automaticamente por toda a cadeia, sem nenhuma chamada manual. É assim que o Angular moderno funciona.

Erros comuns com `effect()`

Evite estas armadilhas ao trabalhar com effects no Angular moderno:

Usar effect para cálculo

Se você precisa derivar um novo valor a partir de outro, use `computed()`. O `effect()` não retorna valores e não deve ser usado como substituto.

Não ler signals dentro do effect

Se o effect não lê nenhum signal, ele nunca será reexecutado. O rastreamento depende da leitura dos signals dentro do callback.

Criar loops infinitos

Se você **escrever** em um signal dentro de um effect que observa esse mesmo signal, você criará um loop infinito. Nunca modifique um signal dentro de um effect que depende dele.

Acessar o navegador sem verificação

Sempre verifique `typeof document !== 'undefined'` antes de acessar APIs do browser para evitar erros em ambientes SSR.

Evoluindo o projeto

Os effects estão funcionando. Agora vamos dar um passo a mais: melhorar a **experiência do usuário** adicionando a funcionalidade de seleção de produto.

Quando o usuário clicar em um produto na lista, o sistema vai registrar qual produto foi selecionado e exibir essa informação na interface. Tudo usando signals e a comunicação pai-filho já estabelecida.

 Agora vamos melhorar o que já temos!



Passo 8 - Criar o estado de seleção

No componente `lista-produtos.ts`, adicione um novo signal para armazenar o produto atualmente selecionado pelo usuário:

```
produtoSelecionado = signal<string | null>(null);
```

O tipo `string | null`

Esse signal pode guardar o nome de um produto (uma `string`) ou `null` quando nenhum produto estiver selecionado. É uma forma segura e expressiva de representar "nenhuma seleção".

O valor inicial `null`

Ao inicializar o componente, nenhum produto está selecionado. Por isso o valor padrão é `null`. Esse estado será alterado quando o usuário interagir com a lista.

Passo 9 - Atualizar o método `exibirProduto`

Agora atualize o método que recebe o evento do componente filho e atualiza o signal de seleção:

```
exibirProduto(nome: string) {  
  this.produtoSelecionado.set(nome);  
}
```

Origem do evento

O componente filho `produto` emite um evento via `@Output` com o nome do produto clicado.

Atualização do estado

O método recebe o nome e chama `.set()` no signal, atualizando o estado do componente pai.

Reatividade automática

Qualquer parte do template ou effect que leia `produtoSelecionado()` será atualizada imediatamente.

Passo 10 - Atualizar o template

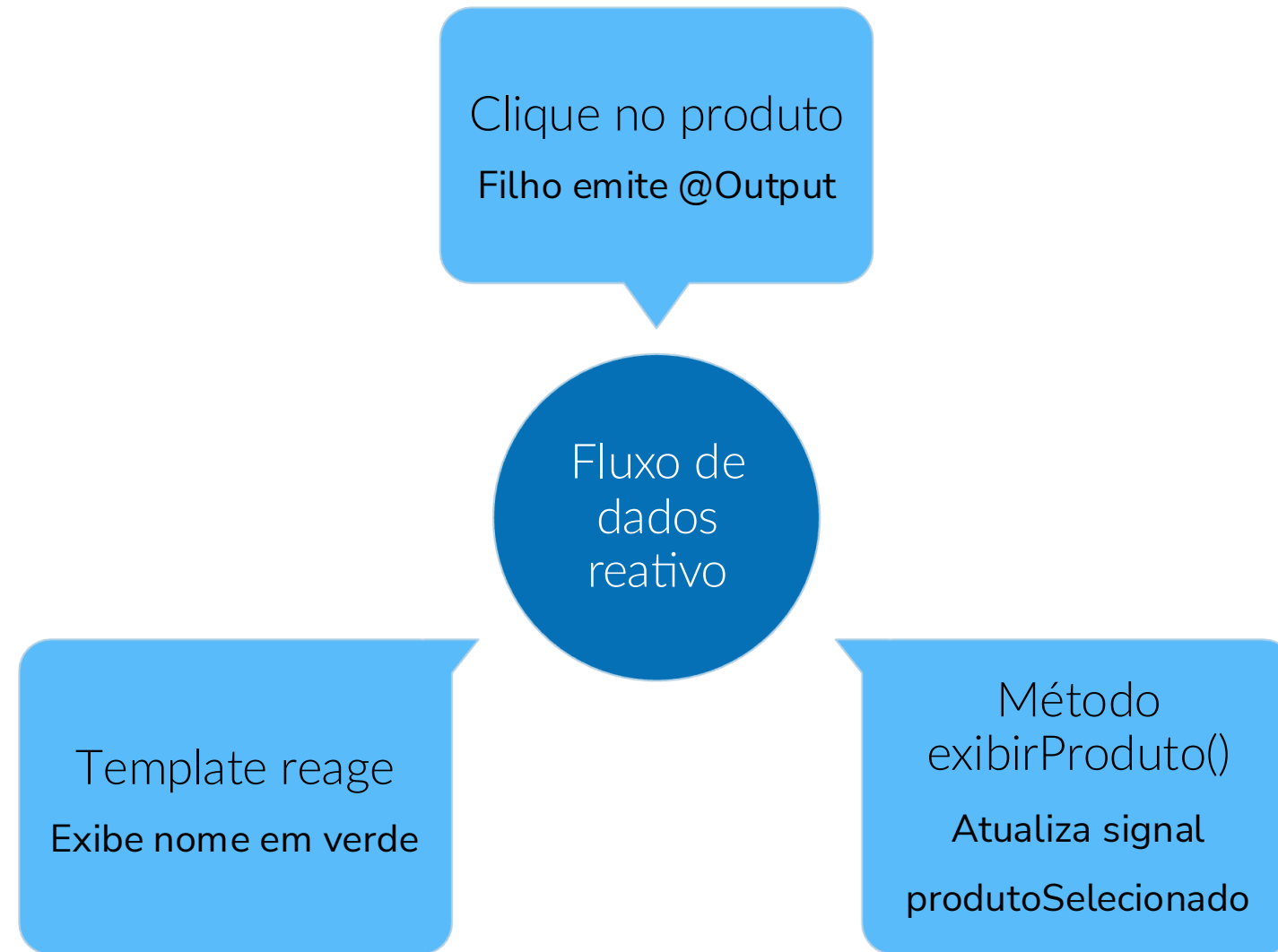
No template `lista-produtos.html`, adicione, no início do arquivo, para exibir o produto selecionado com feedback visual:

```
<h2>
  Produto selecionado:
  @if (produtoSelecionado()) {
    <strong style="color: green;">{{ produtoSelecionado() }}</strong>
  } @else {
    <span style="color: gray;">Nenhum</span>
  }
</h2>
```

- ❏ O `@if` verifica se o signal tem um valor não-nulo. Quando o usuário ainda não selecionou nada, o signal retorna `null`, que é tratado como falso — exibindo "Nenhum" em cinza.

Como tudo se conecta

Veja como o fluxo de dados se move do componente filho até a interface do componente pai:



Essa é a essência da comunicação reativa no Angular moderno: o evento do filho altera o estado do pai, e o template reflete a mudança imediatamente. **Sem nenhum código extra de detecção de mudanças.**



Resultado

1

Usuário clica

O usuário clica em um produto na lista renderizada pelo componente filho.

2

Estado atualizado

O signal `produtoSelecioneado` é atualizado com o nome do produto via `.set()`.

3

Interface reage

O template exibe o nome do produto em verde instantaneamente (sem reload, sem código extra).

Encerramento

O que você conquistou hoje

- Entendeu e aplicou o `effect()` no Angular moderno
- Criou reactions automáticas a mudanças de estado
- Integrou o Angular com APIs do navegador de forma segura
- Implementou seleção de produto com signal e `@if`
- Consolidou os três pilares da reatividade: signal, computed e effect

Sua aplicação agora é

Verdadeiramente reativa. Qualquer mudança de estado propaga-se automaticamente pelo template, pelos valores calculados e pelas ações externas.

Você está construindo Angular do jeito certo.

👉 Base moderna do Angular consolidada. Você está pronto para os próximos desafios do curso!

Para Consulta: Como ficou o arquivo `lista-produtos.ts`:

```
import { Component, signal, computed, effect } from '@angular/core';
import { Produto } from '../produto/produto';

@Component({
  selector: 'app-lista-produtos',
  imports: [Produto],
  templateUrl: './lista-produtos.html',
  styleUrls: ['./lista-produtos.css'],
})

export class ListaProdutos {
  produtos = signal([
    { nome: 'Notebook', preco: 3800 },
    { nome: 'Mouse', preco: 179 }
  ]);

  produtoSelecioneado = signal<string | null>(null);
  totalProdutos = computed(() => this.produtos().length);

  valorTotal = computed(() => {
    return this.produtos()
      .reduce((total, item) => total + item.preco, 0);
  });
```

```
  constructor() {
    effect(() => {
      console.log('Lista de produtos alterada:', this.produtos());
    });

    effect(() => {
      console.log('Valor total atualizado:', this.valorTotal());
    });

    effect(() => {
      if (typeof document !== 'undefined') {
        document.title = `${this.totalProdutos()} Minha Loja`;
      }
    });
  }

  exibirProduto(nome: string) {
    this.produtoSelecioneado.set(nome);
  }
```

```
  adicionarProduto() {
    this.produtos.update(listaAtual => [
      ...listaAtual,
      { nome: 'Teclado', preco: 250 }
    ]);
  }

  substituirProdutos() {
    this.produtos.set([
      { nome: 'Produto novo', preco: 999 }
    ]);
  }
}
```

Para Consulta: Como ficou o arquivo **lista-produtos.html**:

```
<h2>
  Produto selecionado:
  @if (produtoSelecionado()) {
    <strong style="color: green">{{ produtoSelecionado()
  }}</strong>
  } @else {
    <span style="color: gray">Nenhum</span>
  }
</h2>

@for (item of produtos(); track item.nome) {
  <app-produto
    [nome]="item.nome"
    [preco]="item.preco"
    (produtoSelecionado)="exibirProduto($event)"
  ></app-produto>
}
```

```
<p>
  <button (click)="adicionarProduto()">
    Adicionar Produto
  </button>
</p>

<p>
  <button (click)="substituirProdutos()">
    Substituir Lista
  </button>
</p>

<h2>Total de produtos: {{ totalProdutos() }}</h2>

<h3>Valor total: R$ {{ valorTotal() }}</h3>
```

Dúvidas

?

